

Digital Signal Processing with FAUST

Xianglei Wu



BACHELORARBEIT

eingereicht am
Fachhochschul-Bachelorstudiengang

Software Engineering

in Hagenberg

im Mai 2026

Betreuung:

Erik Pitzer, FH-Prof. DI (FH) Dr.

© Copyright 2026 Xianglei Wu

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe <https://creativecommons.org/licenses/by-nc-nd/4.0/>.

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Die vorliegende, gedruckte Arbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Hagenberg, am 25. Mai 2026

Xianglei Wu

Inhaltsverzeichnis

Erklärung	iv
Preface	vii
Abstract	viii
Kurzfassung	ix
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Grundlagen der Audio-DSP	4
2.1.1 Physikalische Grundlage des Audiosignals	4
2.1.2 Abtastung und Quantisierung	4
2.1.3 Signal-Rausch-Verhältnis (SNR)	5
2.1.4 Digitale Audiosignale als Sample-Folgen	5
2.2 Frequenzanalyse mit DFT und FFT	6
2.2.1 Zeitbereich und Frequenzbereich	6
2.2.2 Diskrete Fourier-Transformation	6
2.2.3 Fast Fourier Transformation	7
2.2.4 Amplituden- und Leistungsspektrum	7
2.2.5 Frequenzauflösung	7
2.3 Kurzzeitanalyse und STFT	8
2.3.1 Nichtstationarität von Sprache	8
2.3.2 Frame-basierte Verarbeitung	8
2.3.3 Short-Time Fourier Transform	8
2.3.4 Magnitude und Phase	8
2.4 Fensterfunktionen	8
2.4.1 Blockbildung und Randdiskontinuitäten	8
2.4.2 Spectral Leakage	8
2.4.3 Hanning- und Hamming-Fenster	8
2.5 Overlap-Add und Rekonstruktion	8
2.5.1 Inverse FFT	8

2.5.2	Overlap-Add und Rekonstruktion des Zeitsignals	8
2.6	Echtzeitanforderungen	8
2.6.1	Latenz	8
2.6.2	Puffergröße	8
2.6.3	Rechenzeit pro Frame	8
2.6.4	Overrun und Underrun	8
2.6.5	Echtzeitfähigkeit der spektralen Subtraktion	8
2.7	Zusammenfassung	8
3	Working with LaTeX	9
4	Figures, Tables, Source Code	10
5	Mathematical Elements	11
6	Using Literature	12
7	Printing the Manuscript	13
8	Closing Remarks	14
A	Technical Details	15
B	Supplementary Materials	16
B.1	PDF Files	16
B.2	Media Files	16
C	Questionnaire	17
D	LaTeX Source Code	18
	Quellenverzeichnis	19
	Literatur	19

Preface

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik besonderen Wert in Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik, die mit hohen Anforderungen an Laufzeiteffizienz, deterministische Latenz und numerische Präzision. Grundsätzlich sind die Implementierungen der DSP-Algorithmen in C oder C++, welches aufgrund deren direkten Zugriff auf Hardware Ressourcen und deren extrem effizienten Übersetzer (GCC, Clang) hochgradig optimierte Maschinencode erzeugt, für die Bachelorarbeit die erste Wahl. Der Knackpunkt trotz all den Vorteilen liegt an der doch noch sehr zeitintensive, fehleranfällige mathematische Übersetzung bis hin zu manuelle Implementierung von DSP-Algorithmen und das exportieren auf verschiedenen Zielplattformen, wie auf Desktop, verschiedensten Embedded Systemen, Web und Plug-in-Hosts; Daraus kristallisierten sich in den letzten zwei Jahrzehnten domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung, welches durch Abstraktion und Code-Generierung die Probleme bewältigen soll.

Die domänenspezifische Sprache Faust (Functional Audio Stream), entwickelt am GRAME in Lyon von, verspricht einen Lösungsansatz zu den erwähnten Herausforderungen. Faust ist eine funktionale, blockbasierte Programmiersprache, die speziell für Audio-DSP entworfen wurde. Programme werden als algebraische Komposition von Signalverarbeitungsoperatoren formuliert. Der Faust-Compiler übersetzt die mathematisch knappe Beschreibung eines Signalflussdiagramms in mehrere Zielsprache, darunter C++, Rust, JavaScript und WebAssembly. Aus einer einzigen Quellbeschreibung lassen sich somit Plug-ins für Digital Audio Workstations, Embedded-Anwendungen auf Mikrocontrollern und Webapplikationen erzeugen. [0]

Um für die industrielle Tauglichkeit muss der durch Faust generierte Code sowohl funktional korrekt, als auch laufzeiteffizient sein. Befürworter von Faust argumentieren, dass der generierte Code durch globaler Optimierung über den vollständigen Signalflussgraphen, automatischer Vektorisierung [0] und Parallelisierung [0] mit handgeschriebener C++ Code ebenbürtig oder dieses sogar übertreffen kann. Diese Behauptung vertreten auch die Entwickler in der offiziellen Faust-Dokumentation. [0] Ob handgeschriebener, manuell optimierter C++-Code grundsätzlich performanter ist

als Faust-generierter Code, ist in der wissenschaftlichen Literatur bislang nur punktuell untersucht. Eine systematische Literaturrecherche zeigt zwei peer-reviewte Studien mit direktem Performanz-Vergleich zwischen Faust-generiertem und handgeschriebenem C++-Code.

[0] berichten für den Reverb-Algorithmus *Freeverb* einen Geschwindigkeitsvorteil zugunsten von Faust von 28 Prozent und für den Multi-Tap-Delay *Tapiir* von 49 Prozent. [0] erweitern diesen Vergleich auf neun Instrumente des *Synthesis Toolkit* (STK) und ermitteln dort Geschwindigkeitsvorteile zwischen 18 und 80 Prozent. Beide Studien stammen aus dem erweiterten Faust-Umfeld und untersuchen ausschließlich Algorithmen im Zeitbereich (Reverbs, Delays, physikalische Modelle).

Für FFT-intensive DSP-Algorithmen liegt keine vergleichende Untersuchung vor. Diese Lücke ist nicht zufällig: Die Faust-Dokumentation räumt selbst ein, dass die Sprache durch ihr sample-orientiertes Berechnungsmodell strukturelle Schwierigkeiten mit Multi-Rate-Algorithmen wie FFT und Faltung aufweist [0]. Da gerade FFT-basierte Verarbeitung den Kern vieler praxisrelevanter DSP-Anwendungen –etwa Sprachverbesserung, Audio-Codecs und akustische Analyse –bildet, ergibt sich hier eine wissenschaftlich wie praktisch relevante Forschungslücke.

Die vorliegende Arbeit setzt an dieser doppelten Lücke an: Sie untersucht systematisch und mit differenzierter Methodik, wie sich Faust-generierter C++-Code gegenüber einer manuell implementierten C++-Lösung verhält, wenn beide Implementierungen einen FFT-basierten Algorithmus realisieren. Als Anwendungsfall wird ein klassisches, in der Literatur umfassend dokumentiertes Verfahren der Einkanal-Rauschunterdrückung herangezogen – *Spectral Subtraction* nach Boll. [0] Die Wahl eines etablierten, wohlverstandenen Algorithmus erlaubt es, den Fokus konsequent auf den Sprach- und Implementierungsvergleich zu legen.

1.2 Forschungsfrage und Zielsetzung

Ziel dieser Arbeit ist es an dieser doppelten Lücke anzusetzen. Sie soll die Untersuchung der Eignung von Faust für die Implementierung von FFT-intensiven Echtzeit-DSP-Anwendungen im direkten Vergleich zu einer äquivalenten manuellen nicht optimierten C++-Implementierung zeigen. Dabei soll die Handhabung und Erlernbarkeit der Sprache Faust bei der Implementierung berücksichtigt werden.

Zur Beantwortung der Hauptaufgabe werden folgende Teilfragen untersucht:

- Wie groß ist der Performanz-Unterschied(CPU-Zeit pro Audio-Block) zwischen Faust-generiertem und handgeschriebenem Code für elementare DSP-Bausteine(Gain, Biquad-Filter, FIR, FFT)?
- Wie skaliert dieser Unterschied bei der Verknüpfung zu einer vollständigen Pipeline(Framing, Fensterung, FFT, Spectral Subtraction)?
- Bestehen Objektive Qualitätsunterschiede(PESQ, STOI, SNR-Verbesserung) zwischen den beiden Implementierungen bei identischem Algorithmus?
- Developer Experience: Wie bewährt sich Faust hinsichtlich Entwicklerfreundlichkeit, Codelänge, Wartbarkeit, Komplexität im Vergleich zum Aufbau komplexer Datenstrukturen in C++?

1.3 Aufbau der Arbeit

Die vorliegende Bachelorarbeit ist methodisch in acht Kapitel strukturiert. Der Prozess umfasst, theoretische Grundlagen der DSP, Aufbau des Prototyps, die Durchführung systematischer Vergleiche sowie die abschließende Evaluierung.

Nach dieser Einleitung wird im Kapitel 2 (die Grundlagen) für das notwendige Verständnis der Digitale Signalverarbeitung erläutert, genaueres über Audio-DSP, Fast Fourier Transformation, Fensterfunktionen und sowie generell Echtzeitanforderungen.

Darauf aufbauend stellt Kapitel 3 (Faust) die domänenspezifische Sprache im Detail vor. Einblicke in die funktionale Syntax und Semantik sowie die zugrundeliegende Compiler-Architektur. In Kapitel 4 (Spectral Subtraction) wird die ausgewählte Algorithmus, ein Rauschunterdrückungsverfahren nach Boll theoretisch hergeleitet und auf deren Parametern genauer eingegangen.

In Kapitel 5 (Implementierung) wird die praktische Umsetzung beider Varianten beschrieben. Faust-Code, C++-Code, gemeinsame Schnittstellendesign und sowie Build-System für eine faire Vergleichsbasis werden dokumentiert.

Die Beantwortung der Forschungsfragen erfolgt in Kapitel 6 (Evaluation). In diesem Kapitel werden die Versuchsaufbau detailliert dargelegt und die Ergebnisse der Mikro- und Makro-Benchmarks (Laufzeit, Speicher, Binärgröße) sowie der Qualitätsmessungen (PESQ, STOI) auf beiden Testplattformen ausgewertet.

Danach wird in Kapitel 7 (Diskussion) die erhobene Daten interpretiert. Es werden Limitationen der Untersuchungen, die Aussagekräftigkeit der Durchführungen und die qualitativen Aspekte der Entwicklererfahrung beider Sprachen gegenübergestellt.

Zum Abschluss mit Kapitel 8 (Fazit und Ausblick) wird die Arbeit zusammengefasst und ein Ausblick auf mögliche Erweiterungen, oder Lücken hingewiesen.

Kapitel 2

Grundlagen

2.1 Grundlagen der Audio-DSP

2.1.1 Physikalische Grundlage des Audiosignals

Das Audiosignal ist physikalisch eine sich in einem Medium ausbreitende Druck- und Bewegungsstörung, die durch ihre zeitlichen und räumlichen Schwingungsgrößen beschrieben wird [0].

Die einfachste Darstellung eines Signals im Zeitbereich ist die Sinuswelle,

$$x(t) = A \cdot \sin(\omega t + \phi) \quad (2.1)$$

also eine zeitliche Schwankung des Drucks um einen Mittelwert [0].

Die wichtigsten Größen im Zeitbereich sind:

- **Amplitude** A : die Größe der Schwingung, also die maximale Druckabweichung vom Ruhezustand.
- **Frequenz** f : die Anzahl der Schwingungswiederholungen pro Sekunde, gemessen in Hertz (Hz).
- **Periode** T : die Dauer einer vollständigen Schwingung, mit

$$T = \frac{1}{f} \quad (2.2)$$

- **Phase** ϕ : die Anfangslage der Schwingung im Zyklus, angegeben in Radiant.

2.1.2 Abtastung und Quantisierung

Abtastung auch als Sampling bezeichnet dient dazu aufgrund dessen, dass das analoge Signale zu jedem Zeitpunkt existiert, wird zu bestimmten, gleichmäßigen Zeitpunkten das Signal abgetastet, gemessen. Alles dazwischen wird ignoriert. Die Anzahl der Messungen pro Sekunde ist die Abtastfrequenz f_s gemessen in Hz.

Bei der Abtastung ist die wichtigste Regel das *Nyquist – Theorem*. Sie besagt dass mindestens doppelt so schnell abtasten wie die höchste Frequenz im Signal, sonst kommt es zu Fehlern.

$$f_s \geq 2 \cdot f_{\max} \quad (2.3)$$

[rossing2007springerP521-522].

Quantisierung ist der Schritt nach der Abtastung. Die unendlich genaue Messwerte werden auf einen diskreten Zahlenwert gerundet. Sie wird durch die Bittiefe begrenzt. Durch das Runden wird ein Quantisierungsfehler erzeugt, wobei ein leises Rauschen aus dem Differenz wahrgenommen wird. Je mehr Bits umso besser die Audioqualität [0].

$$x[n] = x(n \cdot T_s), \quad T_s = \frac{1}{f_s} \quad (2.4)$$

2.1.3 Signal-Rausch-Verhältnis (SNR)

SNR beschreibt das Verhältnis der Nutzleistung eines Signals zur Leistung des überlagerten Rauschens. Je höher das SNR, desto weniger wird das Nutzsignal durch Rauschen beeinträchtigt.

Das SNR ist definiert als:

$$\text{SNR} = \frac{P_{\text{Signal}}}{P_{\text{Rauschen}}} \quad (2.5)$$

Da Leistungsverhältnisse sehr große oder sehr kleine Zahlen annehmen können, wird das SNR in der Praxis logarithmisch in Dezibel (dB) angegeben:

$$\text{SNR}_{\text{dB}} = 10 \cdot \log_{10} \left(\frac{P_{\text{Signal}}}{P_{\text{Rauschen}}} \right) \quad (2.6)$$

Sind statt Leistungen die Amplituden bekannt, gilt:

$$\text{SNR}_{\text{dB}} = 20 \cdot \log_{10} \left(\frac{A_{\text{Signal}}}{A_{\text{Rauschen}}} \right) \quad (2.7)$$

Typische Orientierungswerte für die Sprachverarbeitung:

- SNR < 0 dB: Rauschen lauter als Nutzsignal
- SNR ≈ 10 dB: Sprache gerade noch verständlich
- SNR ≈ 20 dB: gute Sprachqualität
- SNR > 40 dB: Studioqualität

Für die spektrale Subtraktion ist das SNR die zentrale Bewertungsgröße: Ziel des Algorithmus ist es, das SNR des verrauschten Eingangssignals $y[n] = x[n] + d[n]$ durch Unterdrückung des Rauschspektrums $D[k]$ zu maximieren, ohne dabei das Nutzsignal $x[n]$ wesentlich zu verzerren. [0]

2.1.4 Digitale Audiosignale als Sample-Folgen

Nach Abtastung und Quantisierung liegt das Audiosignal als diskrete Sample-Folge vor [oppenheim2010]:

$$x[n] = x(n \cdot T_s), \quad n = 0, 1, \dots, N - 1 \quad (2.8)$$

Für die Verarbeitung wird die Folge in überlappende Frames der Länge N unterteilt:

$$x_\lambda[n] = x[n + \lambda \cdot H] \quad (2.9)$$

wobei λ den Frame-Index und H die Hop Size bezeichnen. Diese blockbasierte Struktur ist die direkte Grundlage der Kurzzeit-Fouriertransformation in Abschnitt 2.3.3 [0].

2.2 Frequenzanalyse mit DFT und FFT

2.2.1 Warum brauchen wir den Frequenzbereich? 2.2.2 Was ist die DFT mathematisch? 2.2.3 Wie berechnet man sie effizient? (FFT) 2.2.4 Was steckt im Ergebnis? (Amplitude, Leistung) 2.2.5 Wie genau ist das Ergebnis? (Auflösung)

2.2.1 Zeitbereich und Frequenzbereich

Zum Zweck der Analyse wird gibt es zwei verschiedene Darstellungen in einem Graphen; Im Zeitbereich wird ein Signal als Amplitude über der Zeit dargestellt. Im Frequenzbereich wird dasselbe Signal dagegen als Verteilung seiner Frequenzanteile über der Frequenz in Hertz beschrieben. [tan2018digital] Während der Zeitbereich vor allem in zeitlichen Verlauf zeigt, macht der Frequenzbereich sichtbar, welche Frequenzen im Signal enthalten sind und mit welcher Stärke sie auftreten. Für die Sprachentrauschung ist diese Darstellung besonders wichtig, da sich Sprach- und Rauschteile im Spektrum gezielter analysieren und trennen lassen. [0] Darauf basiert auch die spektrale Subtraktion, bei der ein geschätztes Rauschspektrum vom Spektrum des verrauschten Sprachsignals abgezogen wird. [0]

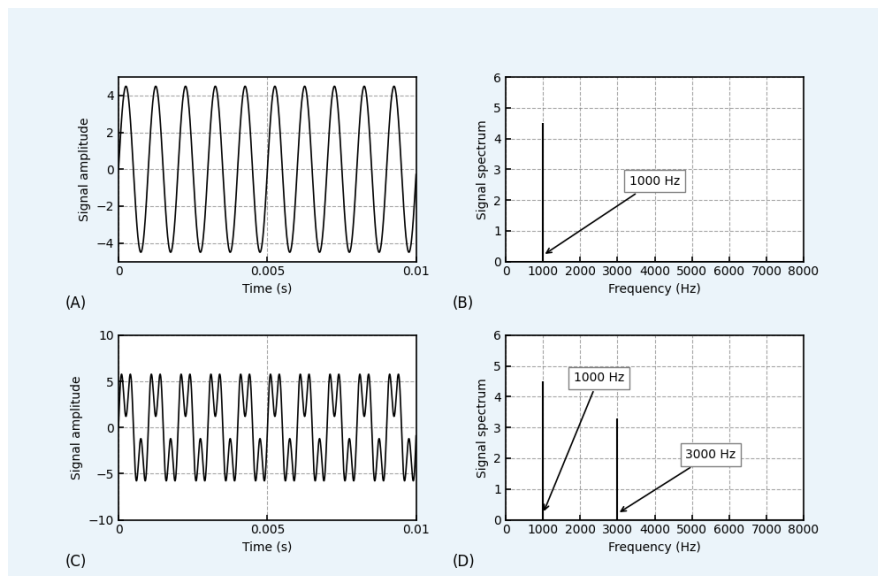


Abbildung 2.1: Zeit und Frequenzbereich Darstellung DSP Signale.

2.2.2 Diskrete Fourier-Transformation

- Definition der DFT-Formel - Erklärung der Variablen (N , k , n , W) - Was $X[k]$ bedeutet
 → komplexe Zahl pro Frequenzbin - Rechenaufwand $O(N^2)$ erwähnen → Motivation für FFT - Inverse DFT kurz erwähnen

2.2.3 Diskrete Fourier-Transformation

Die diskrete Fourier-Transformation (DFT) dient dazu, ein diskretes Signal aus dem Zeitbereich in den Frequenzbereich zu überführen. Dabei werden aus einer Folge mit N Abtastwerten N komplexe DFT-Koeffizienten berechnet [tan2018digital].

Die DFT eines Signals $x[n]$ ist definiert als

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (2.10)$$

Dabei bezeichnet N die Anzahl der betrachteten Abtastwerte, n den Zeitindex beziehungsweise Sample-Index und k den Frequenzindex. Häufig wird der Exponentialterm auch mit dem sogenannten Twiddle-Faktor W_N beschrieben [tan2018digital]. Jeder Koeffizient $X[k]$ ist im Allgemeinen komplex und enthält damit sowohl Betrags- als auch Phaseninformation. Er beschreibt den Anteil einer bestimmten diskreten Frequenzkomponente am Eingangssignal [tan2018digital].

Ein Nachteil der direkten DFT ist ihr hoher Rechenaufwand. Die Berechnung nach der Definition besitzt eine Komplexität von $O(N^2)$ und ist damit für große Datenlängen vergleichsweise aufwendig. Dies ist die Motivation für die Fast Fourier Transformation (FFT), die dieselben DFT-Koeffizienten deutlich effizienter berechnet [tan2018digital].

Zur Rücktransformation in den Zeitbereich wird die inverse diskrete Fourier-Transformation verwendet. Mit ihr lässt sich das ursprüngliche Signal aus den berechneten DFT-Koeffizienten wieder rekonstruieren [tan2018digital].

2.2.4 Fast Fourier Transformation

Hier ist ein direkt einsetzbarer Abschnitt auf Deutsch, in einem LaTeX-nahen Stil formuliert:

2.2.5 Fast Fourier Transformation

Problem: DFT ist zu langsam für Echtzeit

Die diskrete Fourier-Transformation (DFT) ist ein zentrales Verfahren, um ein zeitdiskretes Signal in den Frequenzbereich zu überführen. Das Problem der direkten DFT-Berechnung liegt jedoch im hohen Rechenaufwand: Für eine Datenlänge von

$$N$$

werden

$$N^2$$

komplexe Multiplikationen benötigt. Damit wächst der Aufwand quadratisch mit der Anzahl der Stützstellen, also mit einer Laufzeit von

$$O(N^2)$$

Für kleine Signallängen ist das noch unkritisch. Bei großen Datenmengen, wie sie in der digitalen Signalverarbeitung und insbesondere in zeitkritischen Anwendungen auftreten, wird die direkte DFT jedoch schnell zu teuer. Genau deshalb war ein effizienteres Verfahren notwendig.

Lösung: FFT reduziert den Aufwand auf

$$O(N \log N)$$

Die Fast Fourier Transformation (FFT) ist kein neues Transformationsverfahren, sondern ein schneller Algorithmus zur Berechnung der DFT. Der entscheidende Vorteil besteht darin, dass die Anzahl der notwendigen Operationen drastisch reduziert wird. Für eine Datenlänge von

$$N$$

benötigt die FFT nur etwa

$$\frac{N}{2} \log_2 N$$

komplexe Multiplikationen. Daraus ergibt sich asymptotisch eine Laufzeit von

$$O(N \log N)$$

statt

$$O(N^2)$$

.

Diese Reduktion ist in der Praxis enorm wichtig. Während der Aufwand der DFT mit wachsendem

$$N$$

sehr schnell ansteigt, wächst der Aufwand der FFT wesentlich langsamer. Dadurch wird die Frequenzanalyse auch für große Datenblöcke effizient und in vielen praktischen Fällen erst sinnvoll einsetzbar.

Cooley-Tukey-Algorithmus

Der klassische FFT-Ansatz geht auf den von Cooley und Tukey veröffentlichten schnellen DFT-Algorithmus zurück. Die Grundidee besteht darin, die DFT nicht direkt als ein großes Problem zu berechnen, sondern sie rekursiv in kleinere Teilprobleme zu zerlegen.

Beim Cooley-Tukey-Verfahren wird die Eingangsfolge zunächst in zwei Teilfolgen aufgeteilt, eine mit geraden Indizes und eine mit ungeraden Indizes. Aus diesen beiden Teilfolgen entstehen zwei

$$N/2$$

-Punkt-DFTs, die häufig mit

$$G(n)$$

und

$$H(n)$$

bezeichnet werden. Anschließend werden beide Teilergebnisse mit geeigneten Gewichtungsfaktoren, den sogenannten Twiddle Factors, wieder zur vollständigen DFT zusammengesetzt.

Da

$$N$$

bei der Radix-2-Variante eine Zweierpotenz ist, kann dieser Zerlegungsschritt immer wieder wiederholt werden, bis nur noch sehr kleine Teilprobleme übrig bleiben. Auf diese Weise entsteht die typische mehrstufige FFT-Struktur. Für eine Potenz von 2 gibt es

$$\log_2 N$$

Stufen, wobei pro Stufe nur noch ein linearer Aufwand erforderlich ist. Insgesamt ergibt sich so der deutlich geringere Gesamtaufwand von

$$N \log_2 N$$

komplexen Multiplikationen.

Speedup-Vergleich

Die folgenden Beispielwerte basieren auf den in der Literatur angegebenen Rechenaufwänden

$$N^2$$

für die direkte DFT und

$$\frac{N}{2} \log_2 N$$

für die FFT.

N	DFT-Multiplikationen	FFT-Multiplikationen	Beschleunigung
8	64	12	$5.3 \times$
256	65 536	1 024	$64 \times$
1024	1 048 576	5 120	$204.8 \times$

Tabelle 2.1: Vergleich des Rechenaufwands von DFT und FFT

Besonders anschaulich ist das Beispiel mit

$$N = 8$$

: Für die direkte DFT werden 64 komplexe Multiplikationen benötigt, für die FFT nur 12. Noch deutlicher wird der Unterschied bei

$$N = 1024$$

: Die DFT benötigt 1 048 576 Multiplikationen, die FFT dagegen nur 5 120. Der Geschwindigkeitsvorteil wächst also mit zunehmender Signallänge stark an.

2.2.6 Amplituden- und Leistungsspektrum

- $X[k]$ ist komplex $\rightarrow$ hat Betrag und Phase - Amplitudenspektrum: $|X[k]|$ - Leistungsspektrum: $|X[k]|^2$ - Warum Leistungsspektrum für Spektrale Subtraktion relevant ist - Symmetrie des Spektrums bei reellen Signalen (nur $N/2$ relevant)

$$|X[k]| = \sqrt{\operatorname{Re}(X[k])^2 + \operatorname{Im}(X[k])^2} \quad (2.11)$$

$$|X[k]|^2 = \operatorname{Re}(X[k])^2 + \operatorname{Im}(X[k])^2 \quad (2.12)$$

2.2.7 Frequenzauflösung

- Was bedeutet Frequenzauflösung? → kleinster unterscheidbarer Frequenzabstand -
- Formel: $f = f_s / N$ - Kompromiss: größeres $N \rightarrow$ bessere Auflösung, aber mehr Latenz -
- Beispielrechnung mit typischen Werten - Bedeutung für Spektrale Subtraktion

2.3 Kurzzeitanalyse und STFT

2.3.1 Nichtstationarität von Sprache

2.3.2 Frame-basierte Verarbeitung

2.3.3 Short-Time Fourier Transform

2.3.4 Magnitude und Phase

2.4 Fensterfunktionen

2.4.1 Blockbildung und Randdiskontinuitäten

2.4.2 Spectral Leakage

2.4.3 Hanning- und Hamming-Fenster

2.5 Overlap-Add und Rekonstruktion

2.5.1 Inverse FFT

2.5.2 Overlap-Add und Rekonstruktion des Zeitsignals

2.6 Echtzeitanforderungen

2.6.1 Latenz

2.6.2 Puffergröße

2.6.3 Rechenzeit pro Frame

2.6.4 Overrun und Underrun

2.6.5 Echtzeitfähigkeit der spektralen Subtraktion

2.7 Zusammenfassung

Kapitel 3

Working with LaTeX

Kapitel 4

Figures, Tables, Source Code

Kapitel 5

Mathematical Elements, Equations and Algorithms

Kapitel 6

Using Literature and other Resources

Kapitel 7

Printing the Manuscript

Kapitel 8

Closing Remarks

Anhang A

Technical Details

Anhang B

Supplementary Materials

List of supplementary data submitted to the degree-granting institution for archival storage (in ZIP format).

B.1 PDF Files

Pfad: /

thesis.pdf Master/Bachelor thesis (complete document)

B.2 Media Files

Pfad: /media

*.ai, *.pdf Adobe Illustrator files

*.jpg, *.png raster images

*.mp3 audio files

*.mp4 video files

Anhang C

Questionnaire

Anhang D

LaTeX Source Code

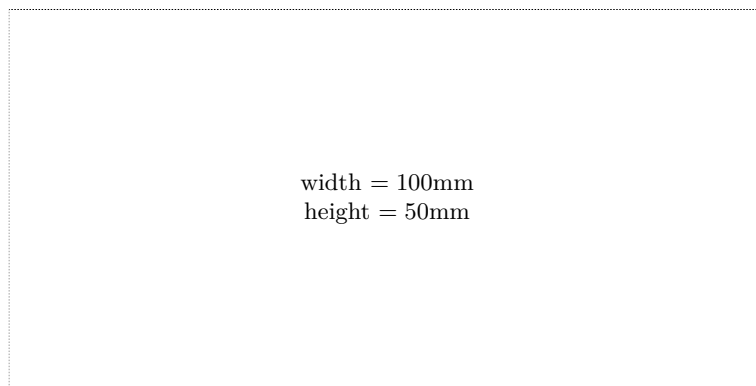
Quellenverzeichnis

Literatur

- [0] Steven Boll. „Suppression of acoustic noise in speech using spectral subtraction“. *IEEE Transactions on acoustics, speech, and signal processing* 27.2 (1979), S. 113–120 (siehe S. 2).
- [0] Stéphane Letz, Yann Orlarey und Dominique Fober. „Work Stealing Scheduler for Automatic Parallelization in Faust“. In: *Proceedings of Linux Audio Conference*. Hrsg. von LAC. Utrecht, Netherlands, 2010. URL: <https://hal.science/hal-02158924> (siehe S. 1).
- [0] Philipos C. Loizou. *Speech Enhancement: Theory and Practice*. 2. Aufl. CRC Press, 2013 (siehe S. 5, 6).
- [0] Romain Michon und Julius O. Smith. „FAUST-STK: A Set of Linear and Nonlinear Physical Models for the FAUST Programming Language“. In: *Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11)*. Paris, France, Sep. 2011 (siehe S. 2).
- [0] Yann Orlarey, Dominique Fober und Stéphane Letz. „FAUST : an Efficient Functional Approach to DSP Programming“. In: *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC*. Hrsg. von Editions DELATOUR FRANCE. 2009, S. 65–96. URL: <https://hal.science/hal-02159014> (siehe S. 1, 2).
- [0] Yann Orlarey, Dominique Fober und Stéphane Letz. „FAUST : an Efficient Functional Approach to DSP Programming“. In: *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC*. Hrsg. von Editions DELATOUR FRANCE. 2009, S. 65–96. URL: <https://hal.science/hal-02159014> (siehe S. 2).
- [0] T. Rossing. *Springer Handbook of Acoustics*. Springer Handbook of Acoustics. Springer New York, 2007. URL: https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 4).
- [0] T. Rossing. *Springer Handbook of Acoustics*. Springer Handbook of Acoustics. Springer New York, 2007. URL: https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 5).
- [0] Nicolas Scaringella u. a. „Automatic vectorization in Faust“. In: *Actes des Journées d’Informatique Musicale JIM2003*. Hrsg. von JIM. Montbeliard, France, 2003. URL: <https://hal.science/hal-02158949> (siehe S. 1).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —